Algorithmic Trading and Quantitative Strategies

Projects

Dr. Ayhan Yuksel, CFA, FDP, FRM, PRM

Bogazici University, EC581

Table of Contents

1. General Issues
2. Factor Investing
3. Statistical Arbitrage
4. Trend Following
5. Forecasting

---

1. General Issues

1.1 Presentation Format

- Each group will be given **30 minutes** to present your work
- All group members are encouraged to present some parts of the presentation

1.2 Presentation Content

First discuss the journal article:

- What is the main trading idea?
- Which methods are used in designing trading strategy? What type of indicators/signals/rules are used?
- Discuss the backtest results. Is it worth investing in such a strategy?

Then present your trading strategy:

- Clearly identify all the steps that you used in strategy development, e.g. construction of indicators/signals/rules
- Include the critical parts of your code into your presentation. Discuss your coding strategy, e.g. do you use custom functions, do you preprocess any data before using Backtrader, etc.
- Use graphs and tables to present your results
- Clearly show the effects of potential improvements that you try, e.g. show the performance of your base strategy without any optimization, and then compare it with the strategy using optimal parameter values, and then walk forward results, etc.
- Compare the main 5-10 performance metrics for your base strategy before any improvements, and the final strategy after using improvements (optimization, money management, etc.)

1.3 General Assumptions for All Projects

- Ignore transaction costs, i.e. `commission=0`
- **Time period:** Use all available data
  - Note that some indicators may not be available for all days (e.g. P/E)
  - You should carefully design your custom functions to properly handle missing input data
- **Initial capital:** 1,000,000 TL
- **Each trade** will have a position size of 100,000 TL
- Define the custom sizer `FixedCashSizer` given below
- In developing your trading strategy, first use **market orders** with `exectype=bt.Order.Market`. To further improve your strategy, you can then try other order types.
- For statistical significance tests, using **Monte Carlo approach**, test whether the Sharpe ratio of your strategy is superior to a random strategy with same constraints and same trading characteristics
  - Calculate p-value for the test
  - Show the histogram of performance statistic for the random portfolios along with your strategy

Custom Sizer: Fixed Cash Per Trade

The following custom sizer allocates a fixed amount of cash (100,000 TL) per trade, regardless of the stock price. This is the Python/Backtrader equivalent of the R `My_OS_Fnc` order-sizing function.

In [ ]:
```python
import backtrader as bt


class FixedCashSizer(bt.Sizer):
    """Allocate a fixed cash amount per trade."""

    params = (('cash_per_trade', 100_000),)

    def _getsizing(self, comminfo, cash, data, isbuy):
        close_price = data.close[0]
        if close_price <= 0:
            return 0
        size = int(self.params.cash_per_trade / close_price)
        return size
```

Usage in a Backtrader strategy:

```
cerebro = bt.Cerebro()
cerebro.broker.setcash(1_000_000)
cerebro.broker.setcommission(commission=0)  # no transaction costs
cerebro.addsizer(FixedCashSizer, cash_per_trade=100_000)
```

1.4 Some Tips on Coding

- When using pandas `shift()` for lagging, be mindful of the sign convention: `shift(1)` shifts data **forward** (i.e. the previous value appears at the current index)
- Parallel processing (e.g. `multiprocessing`) may not work properly for some cases. If you get an error in optimization, try running sequentially
- In walk forward optimizations, some parameter choices may yield very infrequent trading. Try training and test periods that are long enough to generate trades at least for some of the parameter values
- If you will design more than one strategy, use **different script files** for each strategy and save the results of each strategy. If you compare different strategies, use another script for comparison
- Properly comment your code

2. Factor Investing

2.1 Article

A Trend Factor: Any Economic Gains from Using Information over Investment Horizons?

Yufeng Han
UNCC

Guofu Zhou
Washington University in St. Louis - Olin School of Business

Yingzi Zhu
Tsinghua University - School of Economics & Management

June 2016

Abstract:
In this paper, we provide a trend factor that captures simultaneously all three stock price trends: the short-, intermediate-, and long-term, by exploiting information in moving average prices of various time lengths whose predictive power is justified by a proposed general equilibrium model. It outperforms substantially the well-known short-term reversal, momentum, and long-term reversal factors, which are based on the three price trends separately, by more than doubling their Sharpe ratios. During the recent financial crisis, the trend factor earns 0.75% per month, while the market loses −2.03% per month, the short-term reversal factor loses −0.82%, the momentum factor loses −3.88%, and the long-term reversal factor barely gains 0.03%. The performance of the trend factor is robust to alternative formations and to a variety of control variables. From an asset pricing perspective, it also performs well in explaining cross-section stock returns.

Number of Pages in PDF File: 63

Keywords: Trends, Moving Averages, Asymmetric Information, Predictability, Momentum, Factor Models

JEL Classification: G11, G14

2.2 Project

Consider four factors:

ROE Factor

- Used as an example in lecture notes

P/E Factor

- Treat similarly with ROE factor

Momentum Factor

- Define momentum factor as the return of the stock during the past year, i.e.

[\text{Mom}t = \$frac{P_t}{P$\{t-12\}$} - 1]

Trend Factor

- Define trend factor using **predictive regressions** with deviations from moving averages as explanatory variables (as explained in the journal article)
- For this, first using only **BIST100 Index data**, run a predictive regression using all available data. Drop any non-significant variables from the regression equation.
- Then for each month t, for each stock, use the coefficients from the regression equation you found, and the moving averages calculated at time t, predict the next month's return.
- This predicted return will be your factor

In factor construction: discard the outliers and standardize your factors using a **z-score transformation**

Factor Construction Example (Python)

In [ ]:
```
import numpy as np
import pandas as pd
from scipy import stats
```

```python
def compute_momentum_factor(prices: pd.DataFrame, lookback: int = 12) -> pd.DataFrame:
    """Compute momentum factor as trailing return over `lookback` periods."""
    return prices / prices.shift(lookback) - 1


def zscore_transform(factor: pd.DataFrame) -> pd.DataFrame:
    """Cross-sectional z-score with outlier clipping at +/-3."""
    clipped = factor.clip(
        lower=factor.quantile(0.01, axis=1),
        upper=factor.quantile(0.99, axis=1),
        axis=0,
    )
    mu = clipped.mean(axis=1)
    sigma = clipped.std(axis=1)
    return clipped.sub(mu, axis=0).div(sigma, axis=0)
```

2.3 Analysis Tasks

- Explain **why** these factors might work?
- Construct **Factor-Mimicking Portfolios (FMPs)** using both:
  - **Portfolio approach** as shown in lecture slides
  - **Cross-sectional regression approach** as shown in lecture slides
- Compare FMPs
- Analyze factor performance using (raw and rank) **information coefficients**, average monthly returns and **t-test**
  - Which factor earned the highest risk premia in the past?
  - If the starting dates for FMPs differ, compare them using a common starting date
- Compare FMPs based on portfolio vs regression approach. Are they qualitatively similar?
- Pick a date and compare weights for these FMPs for that date (as shown in lecture slides)

---

3. Statistical Arbitrage

3.1 Article

Pairs Trading: Performance of a Relative Value Arbitrage Rule

Evan Gatev
Simon Fraser University

William N. Goetzmann
Yale School of Management - International Center for Finance; National Bureau of Economic Research (NBER)

K. Geert Rouwenhorst
Yale School of Management - International Center for Finance

February 2006

Yale ICF Working Paper No. 08-03

Abstract:
We test a Wall Street investment strategy, pairs trading, with daily data over 1962-2002. Stocks are matched into pairs with minimum distance between normalized historical prices. A simple trading rule yields average annualized excess returns of up to 11 percent for selffinancing portfolios of pairs. The profits typically exceed conservative transaction costs estimates. Bootstrap results suggest that the pairs effect differs from previously-documented reversal profits. Robustness of the excess returns indicates that pairs trading profits from temporary mis-pricing of close substitutes. We link the profitability to the presence of a common factor in the returns, different from conventional risk measures.

Number of Pages in PDF File: 47

JEL Classification: G1

3.2 Project — Pair Construction

- Construct at least **10 pairs** based on ownership, sector, etc.
- In selecting pairs, compare their price trajectories. Ignore any pairs among two stocks if the price series do not seem to be cointegrated

After selecting pairs, construct the relevant information for the pair. Assume that you want to use **GARAN** and **AKBNK** to construct a pair:

1. Calculate their price ratio as:

$$ \text{Ratio}_t = \frac{P^{\text{GARAN}}_t}{P^{\text{AKBNK}}_t} $$

This will be the price of the instrument that you are trading.

2. Set the name of your pair as `GARAN_AKBNK`

3. Assign the relevant information to your variable:

In [ ]:
```python
import pandas as pd


def construct_pair(price1: pd.Series, price2: pd.Series,
                   name1: str, name2: str) -> pd.DataFrame:
    """Construct pair DataFrame with ratio and individual prices."""
    ratio = price1 / price2
    pair_df = pd.DataFrame({
        'Close': ratio,
        'Price1': price1,
        'Price2': price2,
    })
    pair_df.name = f"{name1}_{name2}"
    return pair_df


# Example usage:
# GARAN_AKBNK = construct_pair(garan_close, akbnk_close, 'GARAN', 'AKBNK')
```

3.3 Project — Strategy Design

Design **two** pairs trading strategies:

Distance Approach

- Design an indicator function to calculate **z-score** for the latest observed normalized difference (as shown in lecture slides)
- The length of formation period (h in lecture notes) will be the sole argument for your indicator function. You can optimize this
- This indicator function will use the values in columns (`Price1` , `Price2`) to calculate (on a rolling window basis) the corresponding z-scores
- You will use this indicator (i.e. z-scores) in designing your trade signals

Example strategy: Enter long when z-score crosses -2 from above, and exit long if it crosses -1 from below. Similarly enter short when z-score crosses $+2$ from below, and exit short if it crosses $+1$ from above.

In [ ]:
```python
import numpy as np
import pandas as pd


def distance_zscore(price1: pd.Series, price2: pd.Series,
                    formation_period: int) -> pd.Series:
    """Rolling z-score of the normalized price difference (distance approach)."""
    spread = price1 / price2
    roll_mean = spread.rolling(window=formation_period).mean()
    roll_std = spread.rolling(window=formation_period).std()
    zscore = (spread - roll_mean) / roll_std
    return zscore
```

Cointegration Approach

- Design an indicator function to fit a **linear regression** between two price series (i.e. `Price1` and `Price2`), calculate the latest z-score for the residual (as shown in lecture slides)
- The length of formation period will be the sole argument for your indicator function. You can optimize this
- You will use this indicator (i.e. z-scores) in designing your trade signals

In [ ]:
```python
import numpy as np
import pandas as pd
from numpy.lib.stride_tricks import sliding_window_view


def cointegration_zscore(price1: pd.Series, price2: pd.Series,
                         formation_period: int) -> pd.Series:
    """Rolling z-score of regression residuals (cointegration approach)."""
    zscore = pd.Series(index=price1.index, dtype=float)

    for i in range(formation_period, len(price1)):
        y = price1.iloc[i - formation_period:i].values
        x = price2.iloc[i - formation_period:i].values
        x_with_const = np.column_stack([np.ones(len(x)), x])
        beta = np.linalg.lstsq(x_with_const, y, rcond=None)[0]
```

```
        residuals = y - x_with_const @ beta
        current_resid = price1.iloc[i] - (beta[0] + beta[1] * price2.iloc[i])
        zscore.iloc[i] = (current_resid - residuals.mean()) / residuals.std()

    return zscore
```

3.4 Strategy Evaluation

For each strategy:

- **Optimize** your strategy parameters
- Test whether it's better to **wait for the first reversal**, e.g. apply a second signal where you check whether the absolute value of z-scores decreased during last period (or last few periods)
- Perform **walk forward optimization**
- Perform **statistical significance tests**

Compare the performance of each strategy:

- Which method, **distance or cointegration**, performed better?
- Use performance metrics, such as net P/L, drawdown, Sharpe ratio in your comparison

---

4. Trend Following

4.1 Article

UNIVERSITY OF EXETER
XFi Centre for Finance and Investment

A Momentum Trading Strategy Based on the Low Frequency Component of the Exchange Rate

Richard D.F. Harris
University of Exeter

Fatih Yilmaz
Bank of America

Paper Number: 08/04

August 2008

Abstract

In this paper, we develop a momentum trading strategy based on the low frequency trend component of the spot exchange rate. Using, alternately, kernel regression and the high-pass filter of Hodrick and Prescott (1997), we recover the non-linear trend in the monthly exchange rate and use short-term momentum in this to generate buy and sell signals. The low frequency momentum trading strategy offers greater directional accuracy, higher returns and Sharpe ratios and lower maximum drawdown than traditional moving average rules. Moreover, unlike traditional moving average rules, the performance of the low frequency momentum trading strategy is relatively robust across different time periods, and to the choice of smoothing parameters across a wide range of values.

4.2 Project — Four Trend Following Strategies

Design four different trend following strategies based on:

Strategy 1: Moving Average Crossover (Exponential MA)

- Use exponential moving average crossover signals

Strategy 2: Direction of Moving Average (Exponential MA)

- You can use different methods to calculate the direction of MAs
- The simplest example is the sign of the latest change in MA, i.e. `np.sign(np.diff(MA))`

Strategy 3: HP Filter

- Briefly explain the HP filter in your presentation
- By using the custom function below, design an indicator
- Inputs to the custom function should not have missing values
- The custom function returns a NumPy array. Adjust the function to convert the output to a pandas Series
- Define λ as the argument of your indicator function. You may optimize this value

Strategy 4: Scatter Plot Smoothing (LOWESS)

- Briefly explain scatter plot smoothing (LOWESS) method in your presentation
- By using `lowess()` from `statsmodels.nonparametric.smoothers_lowess`, design an indicator
- The `lowess()` function returns an array where the second column gives the smoothed values. Adjust the function to convert the output to a pandas Series
- The `lowess()` function has a parameter `frac`. Define this as the argument of your indicator function. You may optimize this value

Optimize each of these strategies using BIST100 Index data, not using stock data.

HP Filter Function

In [ ]:
```python
import numpy as np
import pandas as pd


def hp_filter(x, lam=1600):
    """Hodrick-Prescott filter.

    Parameters
    ----------
    x : array-like
        Input series (must not contain NaN).
    lam : float
        Smoothing parameter (lambda).

    Returns
    -------
    np.ndarray
        The trend (smoothed) component.
    """
    x = np.asarray(x, dtype=float)
    n = len(x)
    eye = np.eye(n)
    D = np.diff(eye, n=2, axis=0)  # second difference matrix
    trend = np.linalg.solve(eye + lam * D.T @ D, x)
    return trend
```

LOWESS Indicator Example

In [ ]:
```python
import pandas as pd
from statsmodels.nonparametric.smoothers_lowess import lowess


def lowess_trend(prices: pd.Series, frac: float = 0.1) -> pd.Series:
    """Compute LOWESS smoothed trend.

    Parameters
    ----------
    prices : pd.Series
        Price series.
    frac : float
```

```
        Fraction of data used in local regression (0 < frac <= 1).

    Returns
    -------
    pd.Series
        Smoothed values with the same index as `prices`.
    """
    x_num = np.arange(len(prices))
    smoothed = lowess(prices.values, x_num, frac=frac, return_sorted=False)
    return pd.Series(smoothed, index=prices.index, name='lowess_trend')
```

4.3 Strategy Comparison & Regime Filter

Compare each strategy:

- Compare the equity curves and performance statistics
- Which one yields best results?

Pick one among four available trend following methods.

Use your selected strategy on individual stocks:

- You can just change the symbol list (from BIST to names of all stocks) for the strategy that you picked

Apply a regime filter based on comparison of BIST100 price vs its HP-filtered value:

- Use the same strategy as above for individual stocks
- But this time, you will add a **second signal** where you check:
  - Whether the BIST Index is **above** its HP-filtered value in entering **long** trades
  - Whether the BIST Index is **below** its HP-filtered value in entering **short** trades
- The logic for the second signal is as follows: In the base strategy, we enter long when the individual stock is in an uptrend. In the modified strategy, we enter long when **both** the individual stock **and** BIST are in an uptrend. Similar logic applies for short trades as well.

---

5. Forecasting

5.1 Article

Christoffersen, P.F. and Diebold, F.X. (2006),
"Financial Asset Returns, Direction-of-Change Forecasting, and Volatility Dynamics,"
Management Science, 52, 1273-1287

MANAGEMENT SCIENCE
Vol. 52, No. 8, August 2006, pp. 1273–1287
ISSN 0025-1909 | EISSN 1526-5501 | 06 | 5208 | 1273

DOI 10.1287/mnsc.1060.0520

Financial Asset Returns, Direction-of-Change Forecasting, and Volatility Dynamics

Peter F. Christoffersen
Faculty of Management, McGill University, 1001 Sherbrooke Street West, Montreal, Quebec, Canada H3A 1G5 and
Centre for Interuniversity Research and Analysis on Organizations (CIRANO), Montreal, Quebec, Canada, peter.christoffersen@mcgill.ca

Francis X. Diebold
Department of Economics, University of Pennsylvania, 3718 Locust Walk, Philadelphia, Pennsylvania 19104-6297 and
National Bureau of Economic Research, Cambridge, Massachusetts, fdiebold@wharton.upenn.edu

We consider three sets of phenomena that feature prominently in the financial economics literature: (1) conditional mean dependence (or lack thereof) in asset returns, (2) dependence (and hence forecastability) in asset return signs, and (3) dependence (and hence forecastability) in asset return volatilities. We show that they are very much interrelated and explore the relationships in detail. Among other things, we show that (1) volatility dependence produces sign dependence, so long as expected returns are nonzero, so that one should expect sign dependence, given the overwhelming evidence of volatility dependence; (2) it is statistically possible to have sign dependence without conditional mean dependence; (3) sign dependence is not likely to be found via analysis of sign autocorrelations, runs tests, or traditional market timing tests because of the special nonlinear nature of sign dependence, so that traditional market timing tests are best viewed as tests for sign dependence arising from variation in expected returns rather than from variation in volatility or higher moments; (4) sign dependence is not likely to be found in very high-frequency (e.g., daily) or very low-frequency (e.g., annual) returns; instead, it is more likely to be found at intermediate return horizons; and (5) the link between volatility dependence and sign dependence remains intact in conditionally non-Gaussian environments, for example, with

5.2 Project — Preprocessing

- Use **weekly data**
- Before using Backtrader, perform the following three steps **separately for each stock** to find μ_t, σ_t and $P[\text{sign}(r_{t:t+h}) = 1]$
- For each stock, after finding these three variables, you can merge these with the OHLC prices and use them within Backtrader

5.3 Forecast Conditional Mean μ_t

Forecast conditional mean return based on **divergences from moving averages**.

First estimate the following **predictive regression**:

[r_{t:t+h} = \alpha + \beta_1 , \text{MAD}^{1}*{t} + \ldots + \beta_K , \text{MAD}^{K}*{t} + \varepsilon_t]

where $\text{MAD}_t^k = P_t / \text{MA}_t^k - 1$ represents the deviation from the k-th moving average.

For weekly data use $h = 1$, i.e. we are forecasting the weekly return.

Given the estimated coefficients, the conditional mean forecast will be:

[\hat{\mu}*{t} = \hat{\alpha} + \hat{\beta}_1 , \text{MAD}^{1}*{t} + \ldots + \hat{\beta}_K , \text{MAD}^{K}*{t}]

In [ ]:
```python
import numpy as np
import pandas as pd
from sklearn.linear_model import LinearRegression


def compute_mad(prices: pd.Series, ma_windows: list[int]) -> pd.DataFrame:
    """Compute Moving Average Deviations (MAD) for multiple windows."""
    mads = pd.DataFrame(index=prices.index)
    for k in ma_windows:
        ma = prices.rolling(window=k).mean()
        mads[f'MAD_{k}'] = prices / ma - 1
    return mads
```

```
def forecast_conditional_mean(prices: pd.Series, ma_windows: list[int],
                              h: int = 1) -> pd.Series:
    """Forecast next-period return using predictive regression on MADs."""
    mads = compute_mad(prices, ma_windows)
    forward_ret = prices.pct_change(h).shift(-h)

    df = mads.join(forward_ret.rename('fwd_ret')).dropna()

    X = df[mads.columns].values
    y = df['fwd_ret'].values

    model = LinearRegression().fit(X, y)

    all_mads = mads.dropna()
    mu = pd.Series(model.predict(all_mads.values),
                   index=all_mads.index, name='mu')
    return mu
```

5.4 Forecast Conditional Volatility σ_t Using EWMA

- Briefly explain the **EWMA method** for volatility forecasting in your presentation
- Use Exponentially Weighted Moving Average volatility with $\lambda = 0.97$
- Include a plot of your volatility forecasts in your presentation
- For a day t, the EWMA volatility calculated using the function below will be your best forecast for the next day volatility

In [ ]:
```
import pandas as pd


def ewma_volatility(returns: pd.Series, lam: float = 0.97) -> pd.Series:
    """Compute EWMA volatility forecast.

    Parameters
    ----------
    returns : pd.Series
        Return series.
    lam : float
        Decay factor (lambda). Default 0.97.

    Returns
```

```
    -------
    pd.Series
        EWMA volatility (standard deviation) series.
    """
    # pandas ewm uses `alpha = 1 - span_decay`; here alpha = 1 - lam
    variance = returns.pow(2).ewm(alpha=1 - lam, adjust=False).mean()
    return variance.pow(0.5).rename('sigma')
```

5.5 Logistic Regression for Direction Forecast

Fit a **logistic regression**:

- Briefly explain logistic regression in your presentation
- Our model has the following form:

[P[\text{sign}(r_{t:t+h}) = 1] \sim f!\left(\frac{\mu_t}{\sigma_t}\right)]

Steps:

1. Construct a variable that takes a value of **1** if the weekly return is positive, and **0** otherwise. Use this variable as the **dependent variable**.
2. Construct a variable by dividing your forecast for conditional mean by your forecast for conditional volatility, i.e. μ_t/σ_t. Use this variable as the **explanatory variable**.
3. After fitting the logistic regression, the fitted values (predicted probabilities) will give you the probability of a positive return, i.e. $P[\text{sign}(r_{t:t+h}) = 1]$.
4. Include a **plot** of this probability in your presentation.

In [ ]:
```
import numpy as np
import pandas as pd
from sklearn.linear_model import LogisticRegression


def fit_direction_model(mu: pd.Series, sigma: pd.Series,
                        returns: pd.Series) -> pd.Series:
    """Fit logistic regression: P[sign(r) = 1] ~ f(mu / sigma).

    Returns
```

```
    -------
    pd.Series
        Predicted probability of a positive return.
    """
    signal = (mu / sigma).rename('signal')
    y = (returns > 0).astype(int).rename('direction')

    df = pd.concat([signal, y], axis=1).dropna()

    X = df[['signal']].values
    y_vals = df['direction'].values

    model = LogisticRegression(solver='lbfgs').fit(X, y_vals)

    prob = pd.Series(model.predict_proba(signal.dropna().values.reshape(-1, 1))[:, 1],
                     index=signal.dropna().index, name='prob_positive')
    return prob
```

5.6 Trading Strategy Design

Design a trading strategy:

- If probability of a positive return is **above a certain threshold**, enter long
- If probability of a positive return is **below another threshold**, exit long (i.e. **long only strategy**)

Optimize your parameters:

- Optimize the two probability thresholds mentioned above
- Perform **walk forward optimization**
- Perform **statistical significance tests**

In [ ]:
```
import backtrader as bt


class ForecastStrategy(bt.Strategy):
    """Long-only strategy based on probability of positive return."""

    params = (
```

```
        ('entry_threshold', 0.6),
        ('exit_threshold', 0.4),
    )

    def __init__(self):
        self.prob = self.data.lines.close  # placeholder — replace with probability line

    def next(self):
        if not self.position:
            if self.prob[0] > self.params.entry_threshold:
                self.buy()
        else:
            if self.prob[0] < self.params.exit_threshold:
                self.close()
```

End of Project Descriptions